

FlyRank Backend AI Engineering

Week 3 - Assignment A2: Connecting CRUD to the Database

Student: Anas Abdelmotelb Mansour | **Project:** Task API

Repository: github.com/AnasAbdelmotelb/task-api

1. Objective

The objective of this assignment was to replace the Task API's in-memory storage with a real SQLite database while keeping the existing CRUD API behavior consistent. The implementation uses FastAPI and SQLite and stores tasks persistently in **tasks.db**.

2. Implementation Summary

Area	Completed implementation
Backend	FastAPI / Python
Database	SQLite with automatic database/table initialization
Data	Three example tasks inserted when the database is empty
API	CRUD endpoints preserved; task data stored using SQL queries
Validation	400 Bad Request for invalid task data; 404 for unknown task IDs
Documentation	README with setup, endpoints, SQL examples, validation details, and database screenshot
Submission	Public GitHub repository

3. Evidence

Figure 1. Assignment brief

The screenshot shows the FlyRank assignment brief interface. On the left is a dark sidebar with navigation links: Dashboard, Profile (100%), Onboarding (100%), Anthropic Certs (100%), Tracks, Assignments (selected), Schedule, Events, Completion, Resources, Free Tier, Announcements (40), E-Mail Inbox (36), Community, Bosnia Challenge (New), and Support. The main content area has a header with a user profile (Anas Mansour, Intern) and a list of requirements. Below the requirements is a 'What you should notice' section and a 'Glossary' table.

updated_at
for every task.

Requirements

Done means every box is ticked.

- The API still exposes the same CRUD endpoints as Assignment 1.
- Tasks are stored in SQLite instead of memory.
- Data survives server restarts.
- The database is automatically created if missing.
- The tasks table is automatically created if missing.
- Three example tasks are inserted only on the first run.
- CRUD operations use SQL queries.
- Unknown ids return 404.
- Invalid requests return 400.
- Public GitHub repository updated with README and database screenshot.

What you should notice

By the end of this assignment, the API should feel almost identical to Assignment 1.

The URLs didn't change.

The request bodies didn't change.

The responses didn't change.

Only the implementation changed.

This separation between the **API layer** and the **data layer** is one of the foundations of backend engineering. Once you understand it, moving from SQLite to PostgreSQL, MySQL, SQL Server, or another database later becomes much easier.

Glossary

Word	What it means
Database	A system that stores data permanently, even after a program stops running.
SQLite	A lightweight SQL database stored in a single file. No separate database server is required.

FlyRank Week 3 A2: connecting the existing CRUD API to a database while preserving the API behavior.

Figure 2. SQLite database evidence

The screenshot shows the DB Browser for SQLite interface. On the left is a file explorer showing the project files: .gitignore, README.md, database-screenshot.png, db-browser.png, and main.py. The main area displays the 'task-api / README.md' file. The 'Database Screenshot' section contains text explaining that the application uses SQLite for persistent task storage and that the screenshot shows the 'tasks' table containing three example tasks. Below this text is a screenshot of the DB Browser for SQLite interface showing the 'tasks' table with three rows of data. The table has columns 'id', 'title', and 'description'. The first row has id 1, title 'Learn to fly', and description 'Learn to fly like a bird'. The second row has id 2, title 'Learn to fly', and description 'Learn to fly like a bird'. The third row has id 3, title 'Learn to fly', and description 'Learn to fly like a bird'. The interface also shows a 'total_tasks' table with one row containing the value 3.

task-api / README.md

Preview Code Blame 149 lines (97 loc) · 2.89 KB

Database Screenshot

The application uses SQLite for persistent task storage.

The screenshot below shows the `tasks` table containing the three example tasks:

The screenshot shows the DB Browser for SQLite interface. The 'tasks' table is selected, and the 'SQL' tab is active. The SQL query is: `SELECT COUNT(*) AS total_tasks FROM tasks;`. The results show a single row with the value 3. The interface also shows a 'total_tasks' table with one row containing the value 3.

id	title	description
1	Learn to fly	Learn to fly like a bird
2	Learn to fly	Learn to fly like a bird
3	Learn to fly	Learn to fly like a bird

total_tasks

total_tasks
3

Execution finished without errors.
Rowid: 1 rows returned to client.
AS Time: 0.000000
SELECT COUNT(*) AS total_tasks
FROM tasks;

DB Browser for SQLite was used to inspect the tasks database and verify persistent task records.

Figure 3. API validation evidence

```
~/task-api -- python - uvicorn main:app --reload      ~/task-api -- bash      ~/task-api -- bash      +

789b218..9694d57  main -> main
[(venv) (base) Apples-MacBook-Air:task-api mac$ git push origin main
Everything up-to-date
[(venv) (base) Apples-MacBook-Air:task-api mac$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .DS_Store
    Swagger UI .png
    Terminal showing the API working.png
    W2-A2-Anas Mansour.pdf
    git log --online.png

nothing added to commit but untracked files present (use "git add" to track)
[(venv) (base) Apples-MacBook-Air:task-api mac$ nano README.md
[(venv) (base) Apples-MacBook-Air:task-api mac$ git add README.md
[(venv) (base) Apples-MacBook-Air:task-api mac$ git commit -m "Update README with database screenshot"
[main eb6335e] Update README with database screenshot
1 file changed, 5 insertions(+), 1 deletion(-)
[(venv) (base) Apples-MacBook-Air:task-api mac$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 434 bytes | 434.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/AnasAbdelmotelb/task-api.git
  9694d57..eb6335e  main -> main
[(venv) (base) Apples-MacBook-Air:task-api mac$ nano main.py
[(venv) (base) Apples-MacBook-Air:task-api mac$ cd ~/task-api
[(venv) (base) Apples-MacBook-Air:task-api mac$ git add main.py
[(venv) (base) Apples-MacBook-Air:task-api mac$ git commit -m "Fix validation errors to return 400 Bad Request"
[main 6332043] Fix validation errors to return 400 Bad Request
1 file changed, 8 insertions(+), 1 deletion(-)
[(venv) (base) Apples-MacBook-Air:task-api mac$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 472 bytes | 472.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/AnasAbdelmotelb/task-api.git
  7bc9f0f..6332043  main -> main
[(venv) (base) Apples-MacBook-Air:task-api mac$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .DS_Store
    Swagger UI .png
    Task-api.xobpro
    Terminal showing the API working.png
    W2-A2-Anas Mansour.pdf
    git log --online.png

nothing added to commit but untracked files present (use "git add" to track)
[(venv) (base) Apples-MacBook-Air:task-api mac$ █
```

Terminal testing demonstrates API requests and validation/error-handling behavior during development.

Figure 4. Public GitHub repository

task-api

Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file

Add file

Code

Anas Abdelmotelb Mansour

Fix validation errors to return 400 Bad Request

6332043 · 13 minutes ago

22 Commits

.gitignore	Stage 5: ignore generated SQLite database	4 days ago
README.md	Fix database screenshot in README	2 days ago
database-screenshot.png	Add SQLite database screenshot	2 days ago
db-browser.png	Add SQLite database screenshot to documentation	4 days ago
main.py	Fix validation errors to return 400 Bad Request	13 minutes ago

README

Task API

A simple REST API built with FastAPI for managing tasks.

Features

- Get all tasks
- Get a task by ID
- Create a new task
- Update an existing task
- Delete a task
- Health check endpoint

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors

No contributors

Languages

Python 100%

Suggested workflows

Based on your tech stack

The final public repository contains main.py, README documentation, database screenshots, and the project commit history.

4. Result

The Task API now uses SQLite for persistent storage and retains the required CRUD interface. Database initialization, seeded example tasks, SQL-based operations, validation/error handling, documentation, and public GitHub publication were completed. The repository also includes database evidence in the

README.

5. Submission

Deliverable link: <https://github.com/AnasAbdelmotelb/task-api>

Recommended submission: submit the public GitHub repository link. This short report can be uploaded as additional supporting evidence if desired.